

Class 7

David Majeed (A17885958)

Table of contents

Background	1
Principal Component Analysis (PCA)	8
Bar Plot	10
Pairwise & Heat Map	13
PCA Plots	15

Background

Today we will explore some core machine learning methods that are very popular.

These include **clustering** and **dimensional reduction** K-Means Clustering

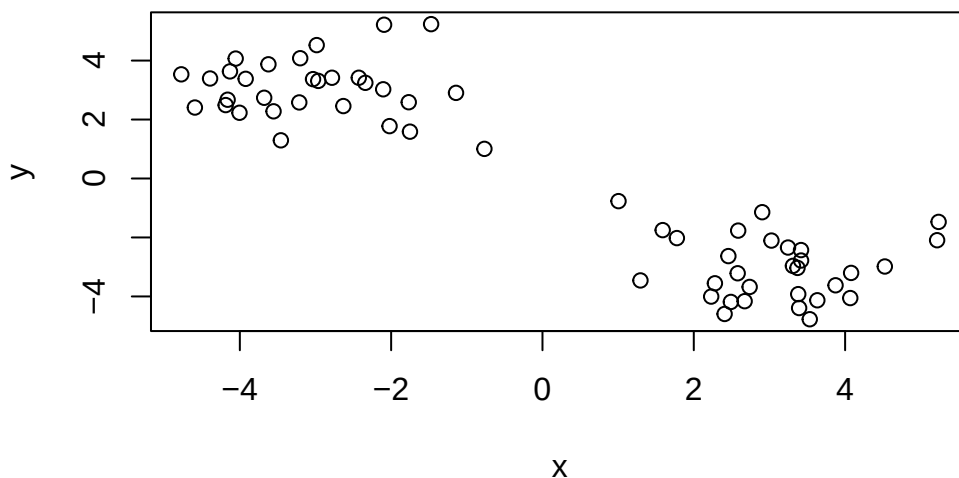
The main function for K-means clustering is `kmeans()` Lets make some simple data with the `rnorm()`

```
x<- c(rnorm(30, -3),  
rnorm(30, 3))  
x
```

```
[1] -2.0935675 -4.0044599 -2.4276426 -2.9842857 -4.5941598 -2.0212304  
[7] -4.1303992 -2.9633935 -3.4575897 -1.1427735 -2.7822462 -3.9229210  
[13] -3.2145275 -4.1870134 -3.6781042 -3.0325465 -1.4719567 -4.0551085  
[19] -3.5544531 -1.7511781 -3.6221136 -1.7675244 -2.6321519 -2.3411787  
[25] -4.1614009 -2.1053268 -0.7677419 -4.3933901 -4.7739866 -3.2021689  
[31]  4.0793196  3.5328391  3.3919927  1.0049712  3.0261338  2.6720739  
[37]  3.2456365  2.4569790  2.5867988  3.8736441  1.5888018  2.2790745  
[43]  4.0679980  5.2358523  3.3708053  2.7382998  2.4900231  2.5799865  
[49]  3.3804955  3.4177965  2.9043724  1.2950004  3.3103637  3.6327512  
[55]  1.7763543  2.4074427  4.5248344  3.4175502  2.2299311  5.2160521
```

```
#c() makes a vector, rnorm generates numbers from a normal distribution

z<- cbind(x=x, y=rev(x))
plot(z)
```



```
#cbind puts the two data sets of x and rev(x) together
```

```
km<- kmeans(z, 2)
#we can now use kmeans, the first input is z which is our data it will
#look at. The 2 is the number of clusters
#We can dive deeper with the attributes() function
attributes(km)
```

```
$names
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"

$class
[1] "kmeans"
```

```
#Q? How many points in each cluster?  
sum(km$cluster==2)
```

```
[1] 30
```

```
sum(km$cluster==1)
```

```
[1] 30
```

```
#Or use this  
km$size
```

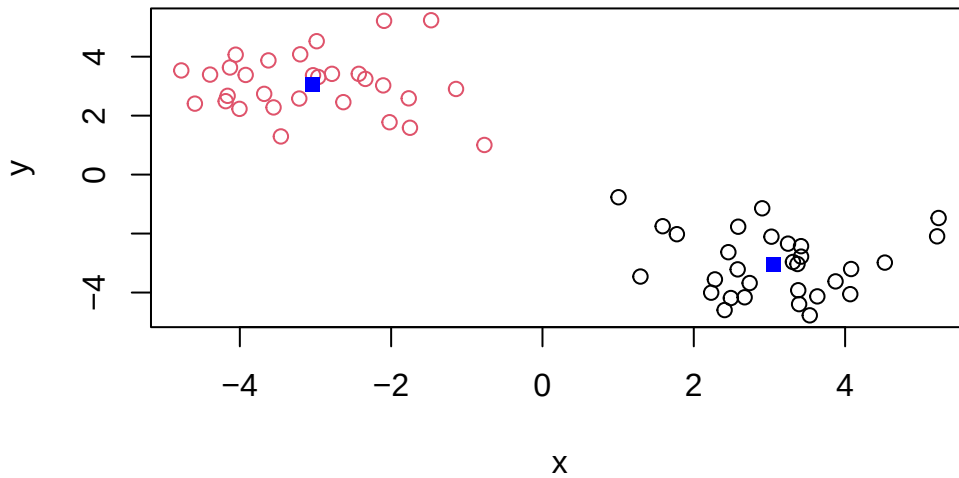
```
[1] 30 30
```

```
#Q? What componet of your result object  
#details cluster assignmnet/membership  
km$cluster
```

```
[1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1  
[39] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

```
#Q? Can we plot our data 'z' and color by kmeans cluster  
#and add cluster centers as blue points
```

```
plot(z,col=km$cluster)  
points(km$centers, col="blue", pch=15)
```



```
#points() put points on the graph and pch=makes the points bigger

#Q? Run a K-means clustering and plot the results asking for 4 cluster

kn<-kmeans(z, 4)
kn
```

K-means clustering with 4 clusters of sizes 16, 12, 18, 14

Cluster means:

	x	y
1	3.197664	-2.217932
2	-1.942043	2.989775
3	-3.774001	3.103160
4	2.897968	-3.982116

Clustering vector:

```
[1] 2 3 2 3 3 2 3 3 3 2 2 3 3 3 3 2 3 3 2 3 2 2 2 3 2 2 3 3 3 1 4 4 1 1 4 1 1
[39] 1 4 1 4 4 1 1 4 4 4 4 1 1 4 1 4 1 4 1 4 1 1 4 1
```

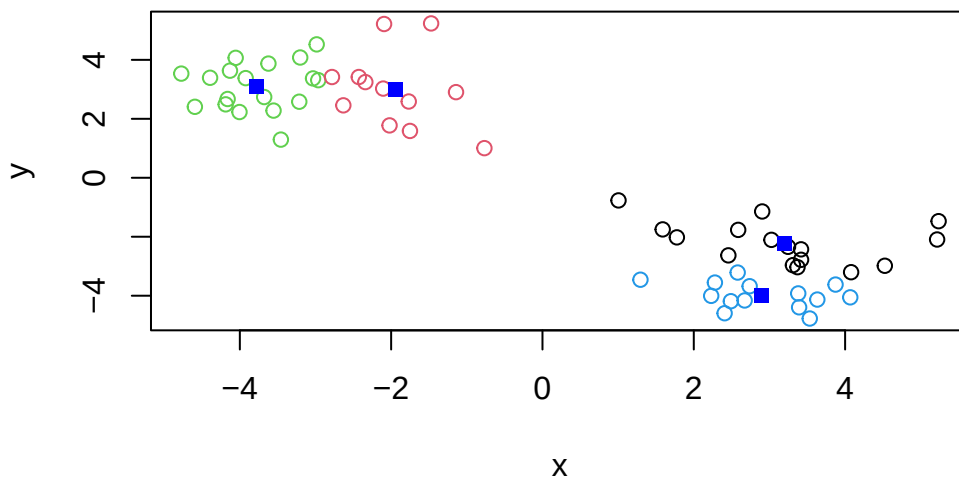
Within cluster sum of squares by cluster:

```
[1] 28.98985 22.20115 16.71165 10.26974
(between_SS / total_SS = 93.7 %)
```

Available components:

[1]	"cluster"	"centers"	"totss"	"withinss"	"tot.withinss"
[6]	"betweenss"	"size"	"iter"	"ifault"	

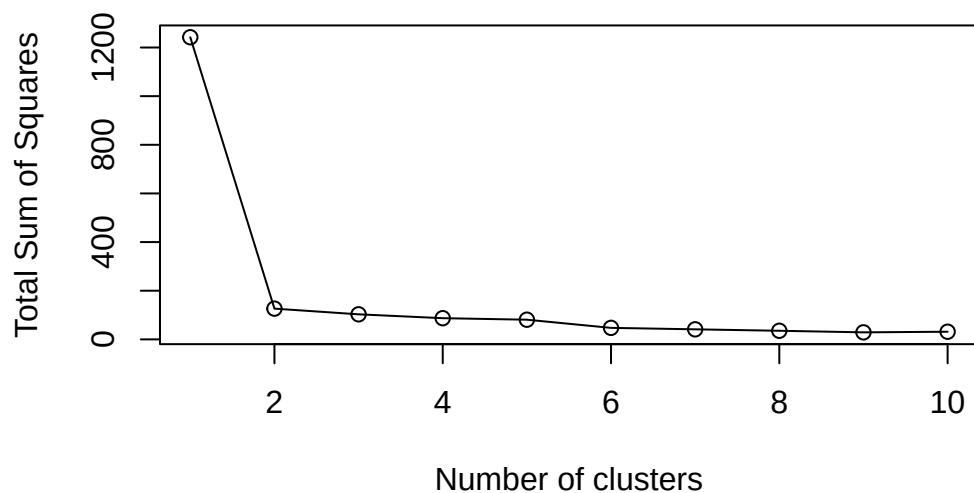
```
plot(z,col=kn$cluster)
points(kn$centers, col="blue", pch=15)
```



One approach is to try different values for center and then pick the best

```
ans<- NULL
for(i in 1:10){
  km<-kmeans(z,i)

  ans<- c(ans, km$tot.withinss)
}
plot(ans, type="o",
      xlab="Number of clusters",
      ylab= "Total Sum of Squares")
```



Hierarchical Clustering

The main function for hierarchical clustering is `hclust()`

The function does not take your “raw” data, you need a distance matrix

```
d<-dist(z)
hc<-hclust(d)
hc
```

Call:

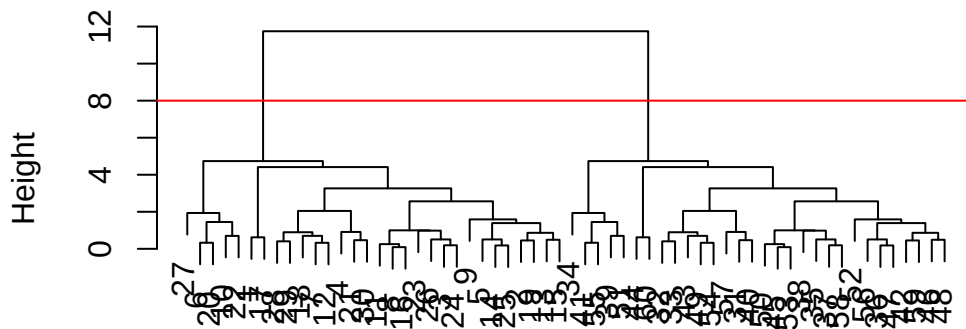
```
hclust(d = d)
```

```
Cluster method   : complete
Distance         : euclidean
Number of objects: 60
```

There is a bespoke `plot()` method for `hclust()` result objects

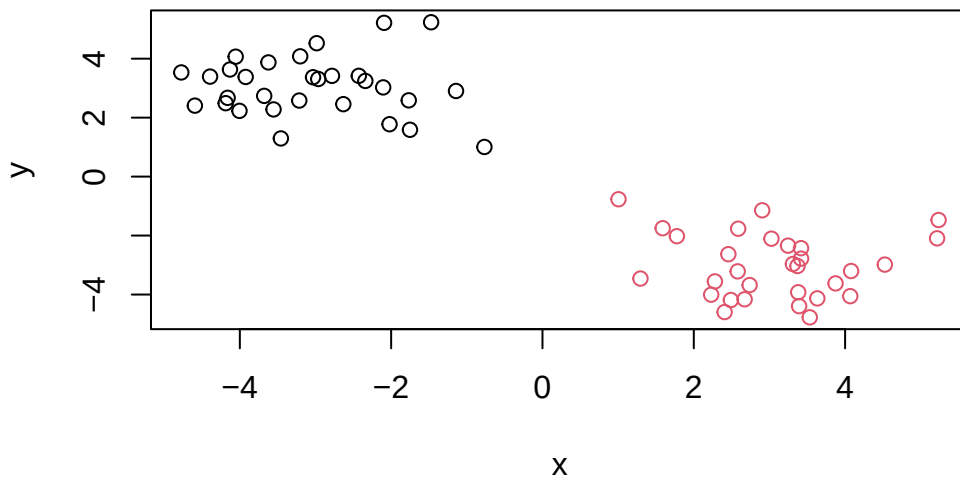
```
plot(hc)
abline(h=8, col="red")
```

Cluster Dendrogram



d
hclust (*, "complete")

```
#This is how we see the cross bars, the abline() draws a line  
#We can cut it to reveal the cluster  
  
grps <- cutree(hc, k=2)  
  
#Q? Make a plot of 'z' with your hclust results (color by cluster membership)  
plot(z, col=grps)
```



Principal Component Analysis (PCA)

PCA is a dimensional reduction method that is popular for revealing patterns in complex datasets

Analysis of UK Food

```
url <- "https://tinyurl.com/UK-foods"  
x <- read.csv(url)  
#load our data from the internet  
  
nrow(x)
```

```
[1] 17
```

```
ncol(x)
```

```
[1] 5
```

```
dim(x)
```

```
[1] 17  5
```

```
#lets find how many rows and columns
```

Q1. 17 rows and 5 columns

```
View(x)
#lets us view the code

#lets set up the rows properly
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url)
rownames(x) <- x[,1]
x <- x[,-1]
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

```
View(x)
dim(x)
```

```
[1] 17  4
```

```
#restate x in order to define it properly for these steps
```

```
x <- read.csv(url, row.names=1)
head(x)
```

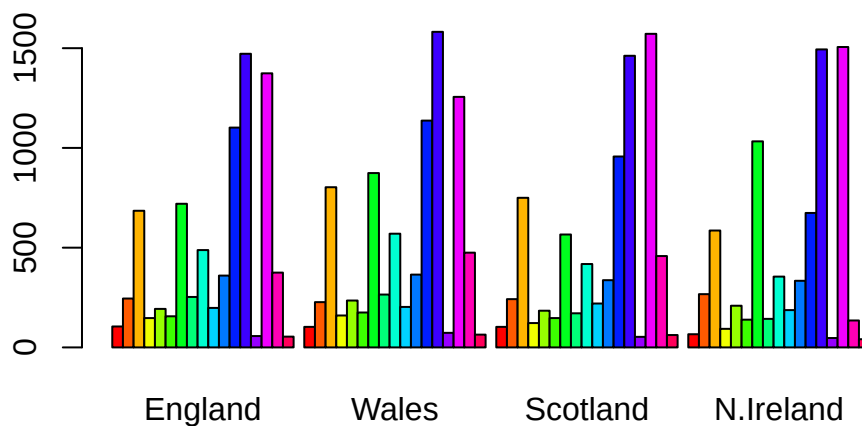
	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

```
#Alt. way to what we did above
```

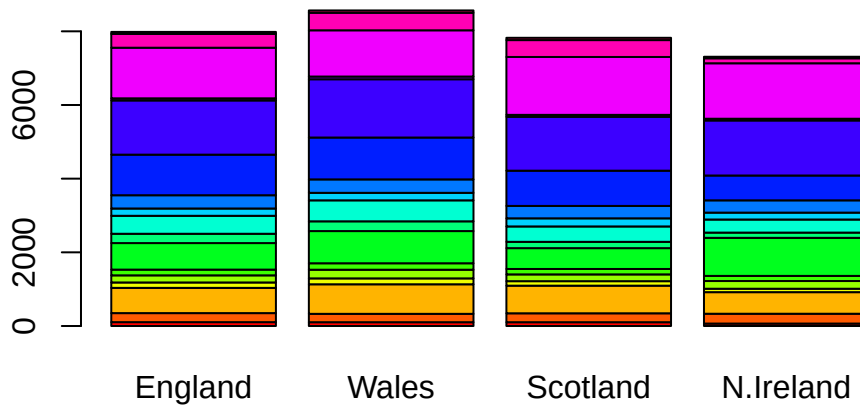
Q.2 I prefer the second method as it is more efficient to code needing less lines and thus leading to less room for mistakes. I was stuck on the first method due to an error in the code which the second method was able to handle much faster. In the circumstance that you do not know how the table looks, the second method is better as it directly goes to the .csv file.

Bar Plot

```
#lets graph it  
barplot(as.matrix(x), beside=T, col=rainbow(nrow(x)))
```



```
#If we want to stack it we can remove the besides=T  
barplot(as.matrix(x), beside=F, col=rainbow(nrow(x)))
```



Q.3 By turning the `besides = false` we can stack it

```
#install.packages("tidyr")
library(tidyr)

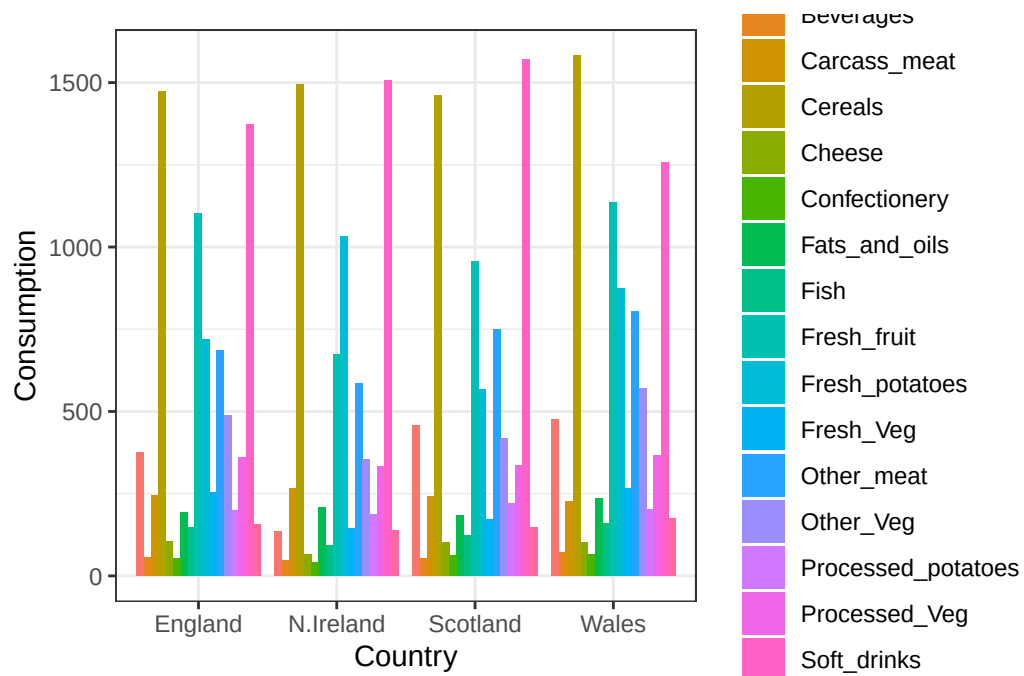
# Convert data to long format for ggplot with `pivot_longer()`
x_long <- x |>
  tibble::rownames_to_column("Food") |>
  pivot_longer(cols = -Food,
               names_to = "Country",
               values_to = "Consumption")

dim(x_long)
```

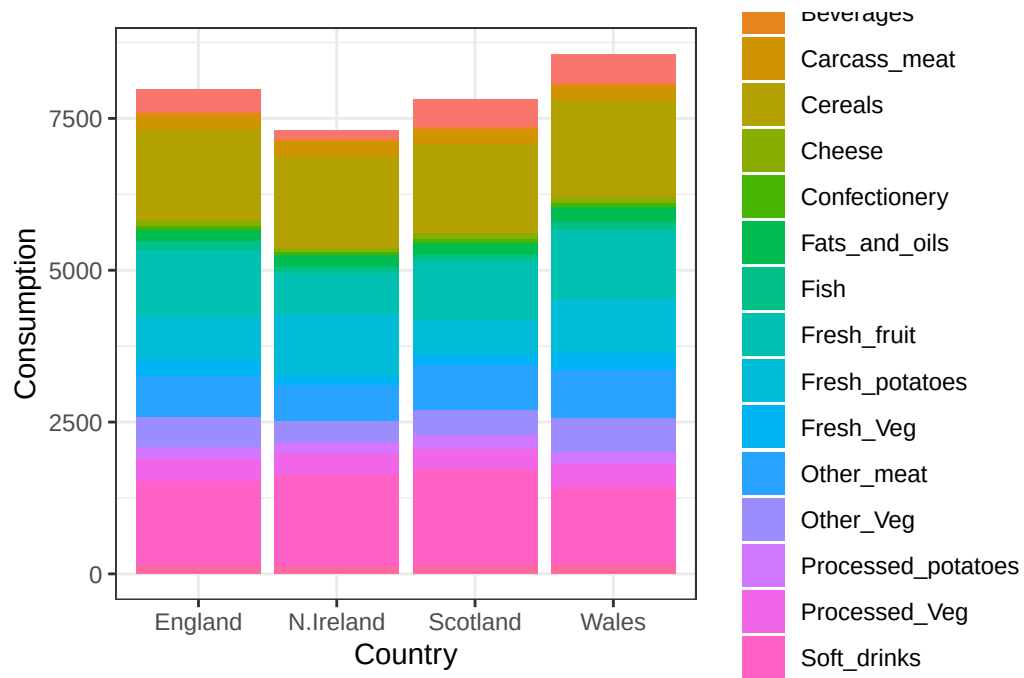
```
[1] 68 3
```

```
#Lets make ggplot for a grouped bar plot
library(ggplot2)

ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "dodge") +
  theme_bw()
```



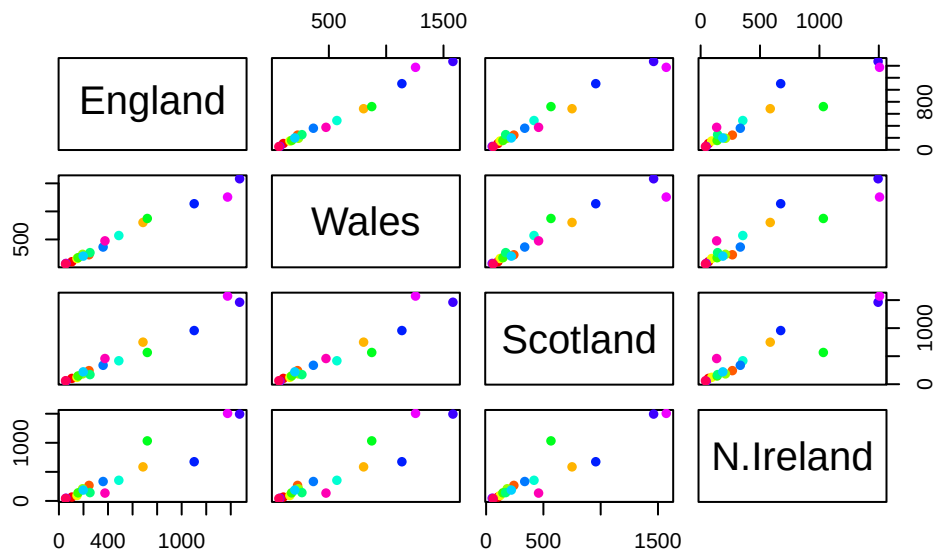
```
#Lets stack it
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col() +
  theme_bw()
```



Q.4 Similarly to the barplot function, we just need to remove a piece of the code to stack it. For ggplot() we remove “position =”dodge” ” which then will stack it

Pairwise & Heat Map

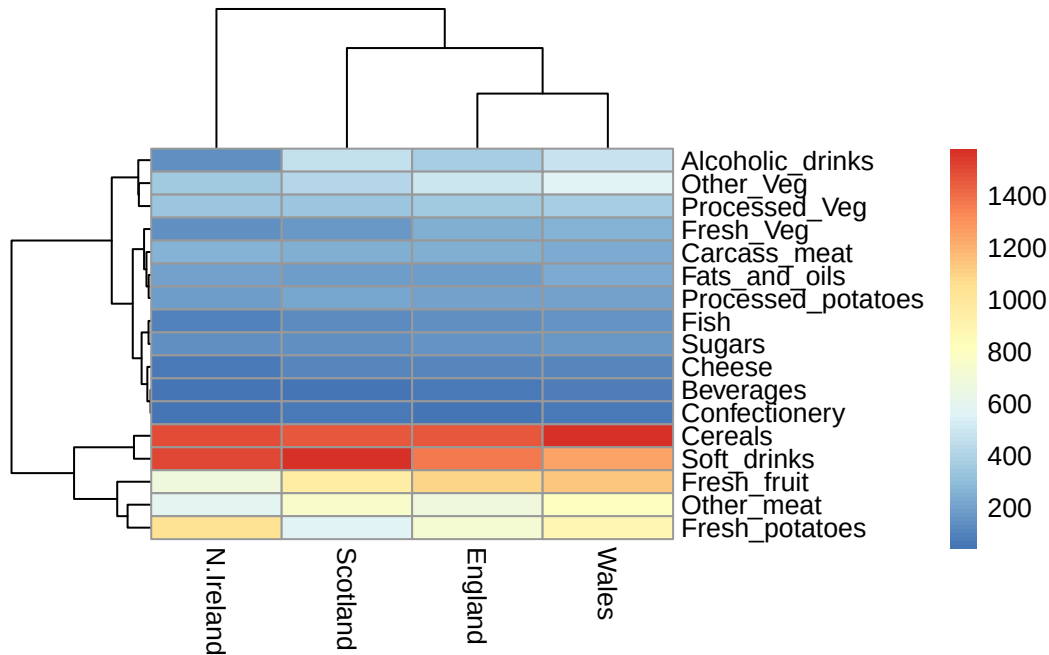
```
pairs(x, col=rainbow(nrow(x)), pch=16)
```



Q.5 Let's make a pairwise plot. This is done using the `pairs()` function, we then use 'x' to signify which data set, and then, we use `col=rainbow(nrow(x))` to color the dots based on the row they come from, and finally, `pch=16` allows us to control the size of the dots. The graph then maps each country to one another and compares them, if they all line up on a diagonal then the data points are similar to one another

```
#lets make some heat maps
#install.packages("pheatmap")
library(pheatmap)

pheatmap( as.matrix(x) )
```



Q.6 From the heatmap, we can see that Wales and England are most similar to one another. The biggest difference between N. Ireland and the rest of the UK is in alcoholic drinks

The main function for PCA is `prcomp()`. The function wants the observations to be the rows and the variables to be the columns

PCA Plots

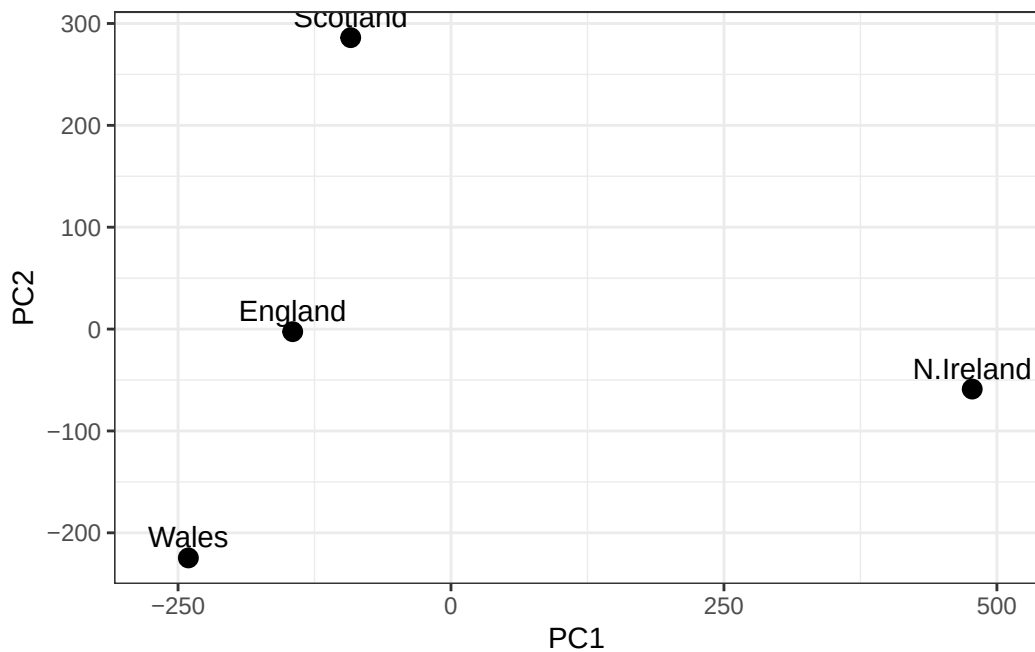
Q.7

```
pca <- prcomp (t(x))

# Create a data frame for plotting
df <- as.data.frame(pca$x)
df$Country <- rownames(df)

# Plot PC1 vs PC2 with ggplot
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) +
  xlab("PC1") +
```

```
ylab("PC2") +  
theme_bw()
```



Q.8

```
t(x)
```

	Cheese	Carcass_meat	Other_meat	Fish	Fats_and_oils	Sugars
England	105	245	685	147	193	156
Wales	103	227	803	160	235	175
Scotland	103	242	750	122	184	147
N.Ireland	66	267	586	93	209	139
	Fresh_potatoes	Fresh_Veg	Other_Veg	Processed_potatoes		
England		720	253	488		198
Wales		874	265	570		203
Scotland		566	171	418		220
N.Ireland		1033	143	355		187
	Processed_Veg	Fresh_fruit	Cereals	Beverages	Soft_drinks	
England		360	1102	1472	57	1374
Wales		365	1137	1582	73	1256
Scotland		337	957	1462	53	1572
N.Ireland		334	674	1494	47	1506

	Alcoholic_drinks	Confectionery
England	375	54
Wales	475	64
Scotland	458	62
N.Ireland	135	41

```
#transposes our data
pca <- prcomp (t(x))
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	3.176e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.000e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.000e+00

#The returned pca objects has components that we can use to make our main result figures

```
attributes(pca)
```

\$names

```
[1] "sdev"      "rotation" "center"   "scale"    "x"
```

\$class

```
[1] "prcomp"
```

#The main results figure from this analysis is called a PC scored plot

```
pca$x
```

	PC1	PC2	PC3	PC4
England	-144.99315	-2.532999	105.768945	-4.894696e-14
Wales	-240.52915	-224.646925	-56.475555	5.700024e-13
Scotland	-91.86934	286.081786	-44.415495	-7.460785e-13
N.Ireland	477.39164	-58.901862	-4.877895	2.321303e-13

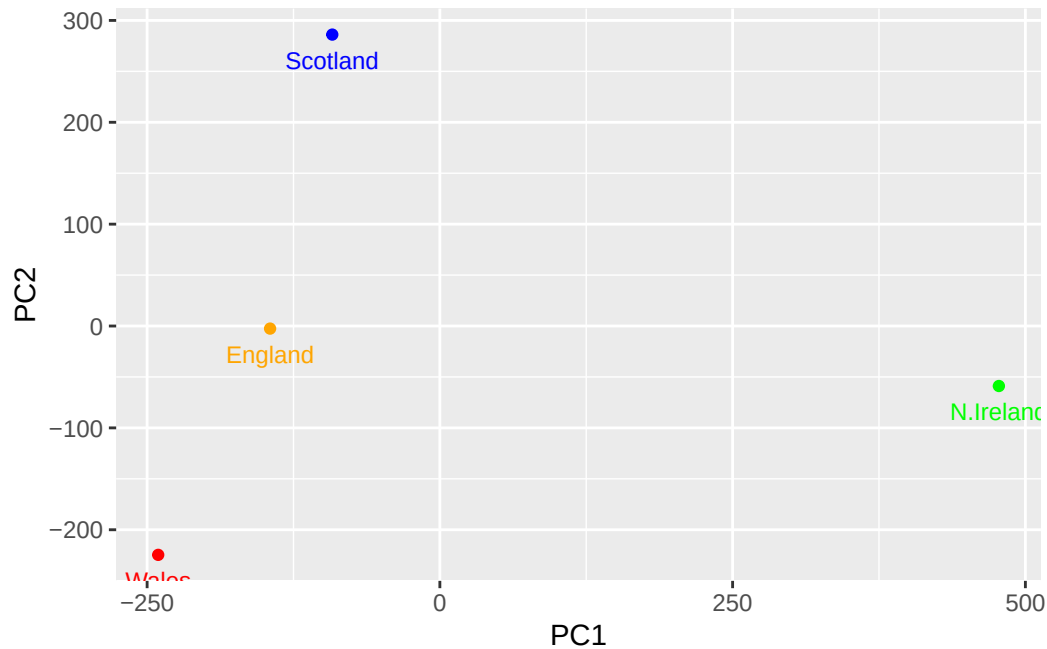
```
my_color<- c("orange","red","blue", "green")
```

```
library(ggplot2)
ggplot(pca$x)+
```

```

aes(PC1,PC2, label=row.names(pca$x))+
geom_point(col=my_color)+
geom_text(size=3,vjust=2, col=my_color)

```

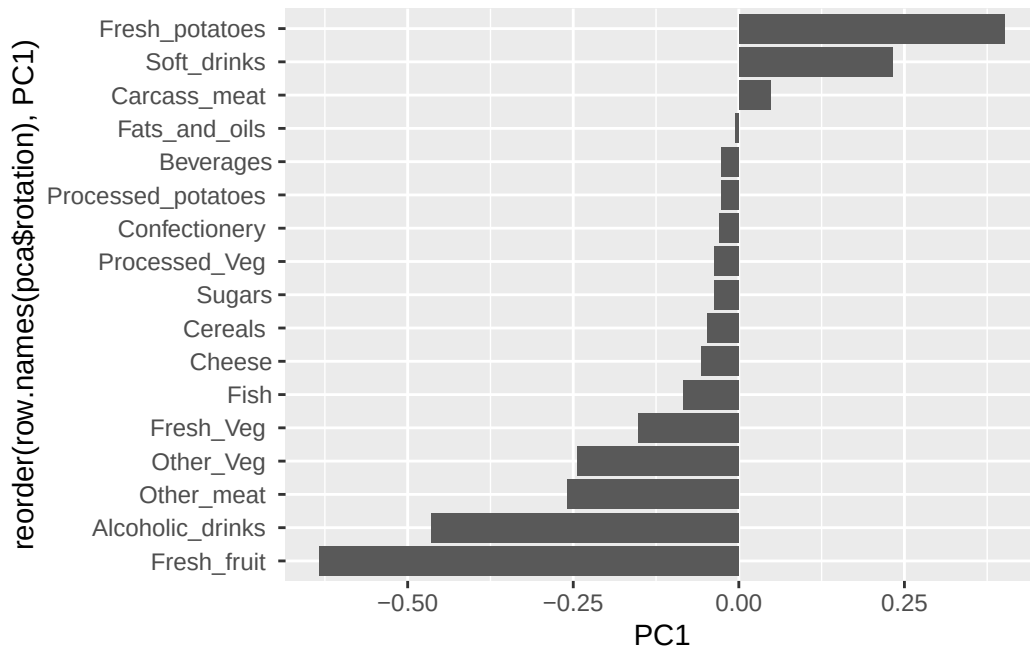


The PC score plot allows us to visually see how the countries vary from one another in a 2d space despite representing 17 data axis. From it we can see that Northern Ireland is the most different than any other country in the United Kingdom.

```

ggplot(pca$rotation)+
aes(PC1, reorder(row.names(pca$rotation),PC1))+
geom_col()

```



The loading plot allows us to see how each of the different categories differ from one another, this is in contrast to the last plot where we were able to see differences among countries but not the categories. This plot can help us find which data axis we should investigate more into.

Q.9. The two food groups are fresh potatoes and soft drinks. PC2 is the axis that spans the second most variation after

```
ggplot(pca$rotation) +
  aes(x = PC2,
      y = reorder(rownames(pca$rotation), PC1)) +
  geom_col(fill = "steelblue") +
  xlab("PC2 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```

